

LAPORAN PRATIKUM PEKAN 6 STRUKTUR DATA

“Double Linked List”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Avero

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “Queue”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 11 Mei 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 13 Mei 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Pratikum.....	1
BAB II PEMBAHASAN	Error! Bookmark not defined.
2.1 QueueArray & QueueArrayDriver.....	Error! Bookmark not defined.
2.2 ReverseData	Error! Bookmark not defined.
2.3 IterasiQueue	Error! Bookmark not defined.
2.4 QueueLinkedList.....	Error! Bookmark not defined.
BAB III KESIMPULAN	9
4.1 Kesimpulan	9
4.2 Saran.....	9
BAB IV DAFTAR PUSTAKA	10
Lampiran Tugas	12
1. Deskripsi Tugas.....	12
2. Aturan Penamaan	12
3. Struktur Program.....	12
4. Contoh Tampilan Output Program.....	Error! Bookmark not defined.
5. Kriteria Penilaian	Error! Bookmark not defined.
6. Deadline	14
Java (ADT) Class · Website_2511531009.....	15
Java (Main/Driver) Class · Browser_2511531009.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya kita telah mengaplikasikan penggunaan struktur data Queue, Stack, dan Linked List. Walaupun demikian, kita menggunakan class-class tersebut karena sudah tersedia pada library bawaan Java. Untuk pemahaman lebih lanjut, kita dapat menggunakan sistem node yang membutuhkan pemahaman mendalam mengenai bagaimana Java mengolah memori untuk dieksekusi.

Salah satu struktur data yang dipelajari adalah Double Linked List (DLL), yaitu pengembangan dari Single Linked List di mana setiap node memiliki dua pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Hal ini memungkinkan penelusuran data dilakukan secara dua arah, baik dari depan ke belakang (forward traversal) maupun dari belakang ke depan (backward traversal), sehingga operasi pengelolaan data menjadi lebih fleksibel dibandingkan Single Linked List.

1.2 Tujuan

1. Memahami konsep Double Linked List.
2. Memahami fungsi node dan pointer (next dan prev) dalam Double Linked List.
3. Memahami operasi dasar Double Linked List pada bahasa pemrograman Java.
4. Memahami proses penelusuran, penambahan, dan penghapusan data pada Double Linked List.

1.3 Manfaat Pratikum

1. Mampu menjelaskan konsep Double Linked List.
2. Mampu menerapkan fungsi node dan pointer (next dan prev) dalam Double Linked List.
3. Mampu mengoperasikan sintaks dasar Double Linked List pada bahasa pemrograman Java.
4. Mampu menerapkan program penelusuran, penambahan, dan penghapusan data pada Double Linked List.

BAB II PEMBAHASAN

2.1 Double Linked List

Double Linked List (DLL) adalah struktur data linear yang terdiri dari sekumpulan node yang saling terhubung. Berbeda dengan Single Linked List, setiap node pada DLL memiliki tiga komponen utama: data (nilai yang disimpan), pointer next (menunjuk ke node berikutnya), dan pointer prev (menunjuk ke node sebelumnya). Node pertama memiliki prev bernilai null, dan node terakhir memiliki next bernilai null.

Struktur DLL memungkinkan traversal dua arah sehingga lebih efisien untuk operasi tertentu seperti penghapusan node tanpa harus menelusuri dari awal. Namun, DLL memerlukan memori lebih besar dibandingkan Single Linked List karena setiap node menyimpan dua pointer.

2.2 Single Linked List Impelentation

2.2.1 NodeDLL

```
1 package pekan6_2511531009;
2
3 public class NodeDLL_2511531009 {
4
5     // mendefinisikan kelas node
6     int data_1009;
7     NodeDLL_2511531009 next_1009;
8     NodeDLL_2511531009 prev_1009;
9
10    public NodeDLL_2511531009(int data_1009) {
11        this.data_1009 = data_1009;
12        this.next_1009 = null;
13        this.prev_1009 = null;
14    }
15 }
16
```

Ini merupakan konstruktor dari yang memiliki class tersendiri agar mudah untuk diakses di class mana saja. Nilai null pada line 12 dan 13 memastikan bahwa setiap node tidak akan menunjuk kepada suatu data random.

2.2.2 insertDLL

```
1 package pekan6_2511531009;
2
3 public class insertDLL_2511531009 {
4
5     // menambahkan node di awal DLL
6     static NodeDLL_2511531009 insertBegin_1009(NodeDLL_2511531009 head_1009, int data_1009) {
7
8         // new node
9         NodeDLL_2511531009 new_node_1009 = new NodeDLL_2511531009(data_1009);
10        new_node_1009.next_1009 = head_1009; // link new node to old head
11        if (head_1009 != null) {
12            head_1009.prev_1009 = new_node_1009;
13        }
14        return new_node_1009;
15    }
16    // func : add node at end
17    public static NodeDLL_2511531009 insertEnd_1009(NodeDLL_2511531009 head_1009, int newData_1009) {
18
19        // new node
20        NodeDLL_2511531009 newNode_1009 = new NodeDLL_2511531009(newData_1009);
21
22        // DLL := null => head
23        if (head_1009 == null) {
24            head_1009 = newNode_1009;
25        }
26        else {
27            NodeDLL_2511531009 curr_1009 = head_1009;
28            while (curr_1009.next_1009 != null) {
29                curr_1009 = curr_1009.next_1009;
30            }
31            curr_1009.next_1009 = newNode_1009;
32            newNode_1009.prev_1009 = curr_1009;
33        }
34        return head_1009;
35    }
36 }
```

insertBegin_1009 yang berfungsi untuk menambahkan node di awal Double Linked List. Method ini menerima dua parameter yaitu head_1009 dan data_1009. Pada line 4 kita membuat node baru dengan constructor NodeDLL_2511531009. Line 5 menghubungkan node baru ke head lama dengan cara mengatur pointer next_1009 dari node baru. Pada line 7-9 kita melakukan pengecekan apakah head_1009 tidak null, jika iya maka pointer prev_1009 dari head_1009 diatur untuk menunjuk ke node baru. Terakhir pada line 10 method ini mengembalikan node baru sebagai head yang baru. Implementasi ini memastikan bahwa DLL tetap terhubung dengan baik baik secara forward maupun backward.

Method insertEnd_1009 berfungsi untuk menambahkan node di akhir Double Linked List. Method ini menerima dua parameter yaitu head_1009 dan newData_1009. Pada line 14 kita membuat node baru dengan data yang diberikan. Pada line 17-18 kita melakukan pengecekan apakah DLL masih kosong (head_1009 null), jika iya maka head_1009 diatur menjadi node baru. Jika tidak kosong, seperti yang terlihat pada line 20-25 kita melakukan iterasi menggunakan pointer curr_1009 untuk mencari node terakhir. Setelah ditemukan, pada line 26-27 kita menghubungkan node terakhir dengan node baru. Method ini mengembalikan head_1009 yang mungkin tetap sama jika DLL tidak kosong.

```

37 // func : add node in certain position
38 public static NodeDLL_2511531009 insertAtPosition(NodeDLL_2511531009 head_1009, int pos_1009, int newData_1009) {
39
40     // newNode
41     NodeDLL_2511531009 newNode_1009 = new NodeDLL_2511531009(newData_1009);
42     if (pos_1009 == 1) {
43         newNode_1009.next_1009 = head_1009;
44         if (head_1009 != null) {
45             head_1009.prev_1009 = newNode_1009;
46         }
47         head_1009 = newNode_1009;
48         return head_1009;
49     }
50     NodeDLL_2511531009 curr_1009 = head_1009;
51
52     for (int i_1009 = 1; i_1009 < pos_1009 - 1 && curr_1009 != null; ++i_1009) {
53         curr_1009 = curr_1009.next_1009;
54     }
55     if (curr_1009 == null) {
56         System.out.println("Posisi tidak ada");
57         return head_1009;
58     }
59     newNode_1009.prev_1009 = curr_1009;
60     newNode_1009.next_1009 = curr_1009.next_1009;
61     curr_1009.next_1009 = newNode_1009;
62     if (newNode_1009.next_1009 != null) {
63         newNode_1009.next_1009.prev_1009 = newNode_1009;
64     }
65     return head_1009;
66 }
67
68 public static void printList_1009(NodeDLL_2511531009 h_1009) {
69     NodeDLL_2511531009 c_1009 = h_1009;
70     while (c_1009 != null) {
71         System.out.print(c_1009.data_1009);
72         if (c_1009.next_1009 != null) { // only print <-> if there's a next node
73             System.out.print(" <-> ");
74         }
75         c_1009 = c_1009.next_1009;
76     }
77     System.out.println();
78 }
79

```

method `insertAtPosition` yang berfungsi untuk menambahkan node pada posisi tertentu dalam Doubly Linked List. Method ini menerima tiga parameter yaitu `head_1009`, `pos_1009`, dan `newData_1009`. Pada line 32 kita membuat node baru dengan data yang diberikan. Pada line 34-39 kita menangani kasus khusus ketika posisi yang diminta adalah 1 (awal DLL). Pada line 41-43 kita menggunakan pointer `curr_1009` untuk berpindah ke posisi yang diinginkan. Pada line 45-46 kita memeriksa apakah posisi yang diminta valid. Jika valid, seperti pada line 48-53 kita mengatur pointer `prev_1009` dan `next_1009` dari node baru dan node di sekitarnya.

method `printList_1009` yang berfungsi untuk menampilkan seluruh elemen dalam Doubly Linked List. Method ini menerima satu parameter yaitu `h_1009` yang merupakan head dari DLL. Pada line 57 kita inisialisasi pointer `c_1009` untuk melakukan traversal dimulai dari head. Pada line 58-63 kita menggunakan loop `while` untuk menelusuri seluruh DLL hingga mencapai akhir. Di dalam loop, pada line 59 kita mencetak data dari node saat ini, kemudian pada line 60-62 kita mencetak simbol "`<->`" hanya jika terdapat node berikutnya.

```

80 public static void main(String[] args) {
81
82     // membuat dll 2 <-> 3 <-> 5
83     NodeDLL_2511531009 head_1009 = new NodeDLL_2511531009(2);
84     head_1009.next_1009 = new NodeDLL_2511531009(3);
85     head_1009.next_1009.prev_1009 = head_1009;
86     head_1009.next_1009.next_1009 = new NodeDLL_2511531009(5);
87     head_1009.next_1009.next_1009.prev_1009 = head_1009.next_1009;
88
89     // cetak DLL awal
90     System.out.print("DLL Awal : ");
91     printList_1009(head_1009);
92
93     // add 1 in the start/at the head
94     head_1009 = insertBegin_1009(head_1009, 1);
95     System.out.print("simpul 1 ditambah di awal : ");
96     printList_1009(head_1009);
97
98     // add 6 at the end
99     System.out.print("simpul 1 ditambah di akhir : ");
100    int data_1009 = 6;
101    head_1009 = insertEnd_1009(head_1009, data_1009);
102    printList_1009(head_1009);
103
104    // add node 4 in pos 4
105    System.out.print("tambah node 4 di posisi 4 : ");
106    int data2_1009 = 4;
107    int pos_1009 = 4;
108    head_1009 = insertAtPosition(head_1009, pos_1009, data2_1009);
109    printList_1009(head_1009);
110 }
111 }
112

```

```

DLL Awal : 2 <-> 3 <-> 5
simpul 1 ditambah di awal : 1 <-> 2 <-> 3 <-> 5
simpul 1 ditambah di akhir : 1 <-> 2 <-> 3 <-> 5 <-> 6
tambah node 4 di posisi 4 : 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6

```

2.2.3 PenelusuranDLL

```
1 package pekan6_2511531009;
2
3 public class PenelusuranDLL_2511531009 {
4
5     // func : forward search
6     static void forwardTraversal_1009(NodeDLL_2511531009 head_1009) {
7         // start search from head
8         NodeDLL_2511531009 curr_1009 = head_1009;
9
10        // continue search until end
11        while (curr_1009 != null) {
12            // print data
13            System.out.print(curr_1009.data_1009 + " <-> ");
14
15            // move to next node
16            curr_1009 = curr_1009.next_1009;
17        }
18        // print a line
19        System.out.println();
20    }
21    // func : backward search
22    static void backwardTraversal_1009(NodeDLL_2511531009 tail_1009) {
23        // start search from the end
24        NodeDLL_2511531009 curr_1009 = tail_1009;
25
26        // continue until the head
27        while (curr_1009 != null) {
28            // print data
29            System.out.print(curr_1009.data_1009 + " <-> ");
30
31            // move to prev node
32            curr_1009 = curr_1009.prev_1009;
33        }
34        // print a line
35        System.out.println();
36    }
37 }
```

forwardTraversal_1009 yang berfungsi untuk menelusuri DLL dari depan ke belakang. Method ini menerima parameter head_1009. Pada line 6 kita inisialisasi curr_1009 dengan head_1009, jadi kita mulai dari kepala DLL. Pada line 9 kita pakai while loop untuk terus berjalan selama curr_1009 tidak null, artinya selama masih ada node yang bisa ditelusuri. Di dalam loop, pada line 11 kita cetak data dari node saat ini dengan format yang ada tanda "<->"-nya. Terus pada line 14 kita geser curr_1009 ke node berikutnya dengan curr_1009.next_1009. Ini penting karena kalau lupa menggeser, nanti program akan stuck di node yang sama terus dan jadi infinite loop. Setelah selesai menelusuri semua node, pada line 17 kita cetak new line biar rapi.

Begitu pula dengan backwardTraversal_1009 yang berfungsi untuk menelusuri DLL dari belakang ke depan. Method ini menerima parameter tail_1009. Pada line 23 kita inisialisasi curr_1009 dengan tail_1009, jadi kita mulai dari ekor DLL. Pada line 26 kita pakai while loop seperti sebelumnya, tapi kali ini kita akan berjalan mundur. Di dalam loop, pada line 28 kita cetak data seperti yang tadi, tapi yang ini akan muncul dari belakang. Yang beda ada pada line 31, kita geser curr_1009 ke node sebelumnya dengan curr_1009.prev_1009. Ini kuncinya, karena DLL itu punya prev dan next, jadi kita bisa maju mundur sesuka hati. Kalau di single linked list kan nggak bisa begini, cuma bisa maju aja.

```

38 public static void main(String[] args) {
39     // print DLLs
40     NodeDLL_2511531009 head_1009 = new NodeDLL_2511531009(1);
41     NodeDLL_2511531009 second_1009 = new NodeDLL_2511531009(2);
42     NodeDLL_2511531009 third_1009 = new NodeDLL_2511531009(3);
43
44     head_1009.next_1009 = second_1009;
45     second_1009.prev_1009 = head_1009;
46     second_1009.next_1009 = third_1009;
47     third_1009.prev_1009 = second_1009;
48
49     System.out.println("Penelusuran Maju : ");
50     forwardTraversal_1009(head_1009);
51
52     System.out.println("Penelusuran Mundur : ");
53     backwardTraversal_1009(third_1009);
54 }
55 }
56
Penelusuran Maju :
1 <-> 2 <-> 3 <->
Penelusuran Mundur :
3 <-> 2 <-> 1 <->

```

2.2.4 HapusDLL

```

1 package pekan6_2511531009;
2
3 public class HapusDLL_2511531009 {
4
5     // func : delete node at head
6     @SuppressWarnings("unused")
7     public static NodeDLL_2511531009 delHead_1009(NodeDLL_2511531009 head_1009) {
8         if (head_1009 == null) {
9             return null;
10        }
11        NodeDLL_2511531009 temp_1009 = head_1009;
12        head_1009 = head_1009.next_1009;
13        if (head_1009 != null) {
14            head_1009.prev_1009 = null;
15        }
16        return head_1009;
17    }
18
19    // func : delete node at the end
20    public static NodeDLL_2511531009 delLast_1009(NodeDLL_2511531009 head_1009) {
21        if (head_1009 == null) {
22            return null;
23        }
24        if (head_1009.next_1009 == null) {
25            return null;
26        }
27        NodeDLL_2511531009 curr_1009 = head_1009;
28        while (curr_1009.next_1009 != null) {
29            curr_1009 = curr_1009.next_1009;
30        }
31        // update previous node pointer
32        if (curr_1009.prev_1009 != null) {
33            curr_1009.prev_1009.next_1009 = null;
34        }
35        return head_1009;
36    }
37 }

```

delHead_1009 dapat menghapus node di awal DLL. Method ini menerima parameter head_1009. Pertama, kita cek apakah head null (DLL kosong), jika iya langsung return null. Kalau tidak kosong, kita geser head ke node berikutnya dengan head_1009 = head_1009.next_1009. Jika head baru tidak null, kita set prev_1009-nya menjadi null karena sekarang jadi node pertama. Terakhir, kita return head yang baru. Ini cukup simple karena hanya perlu mengatur pointer head tanpa perlu traversal.

delLast_1009 yang berfungsi untuk menghapus node di akhir DLL. Method ini menerima parameter head_1009. Kita cek apakah DLL kosong atau hanya memiliki satu node, jika iya return null. Jika tidak, kita traversal ke node terakhir dengan while loop. Setelah sampai di node terakhir, kita update pointer next_1009 dari node sebelumnya menjadi null, sehingga node terakhir terputus dari DLL. Yang perlu diingat, kita tidak perlu mengatur prev_1009 karena node terakhir sudah tidak terpakai.

```

38 // func : delete at certain position
39 public static NodeDLL_2511531009 delPos_1009(NodeDLL_2511531009 head_1009, int pos_1009) {
40 // DLL isEmpty return value
41 if (head_1009 == null) {
42 return head_1009;
43 }
44 NodeDLL_2511531009 curr_1009 = head_1009;
45
46 // transverse until node taht will be deleted
47 for (int i_1009 = 1; curr_1009 != null && i_1009 < pos_1009; ++i_1009) {
48 curr_1009 = curr_1009.next_1009;
49 }
50
51 // if the pos unsearchable
52 if (curr_1009 == null) {
53 return head_1009;
54 }
55
56 // Update pointer
57 if (curr_1009.prev_1009 != null) {
58 curr_1009.prev_1009.next_1009 = curr_1009.next_1009;
59 }
60 if (curr_1009.next_1009 != null) {
61 curr_1009.next_1009.prev_1009 = curr_1009.prev_1009;
62 }
63
64 // if head want to be deleted
65 if (head_1009 == curr_1009) {
66 head_1009 = curr_1009.next_1009;
67 }
68 return head_1009;
69 }
70
71 // func : print DLL
72 public static void printList(NodeDLL_2511531009 head_1009) {
73 NodeDLL_2511531009 curr_1009 = head_1009;
74 while (curr_1009 != null) {
75 System.out.print(curr_1009.data_1009 + " ");
76 curr_1009 = curr_1009.next_1009;
77 }
78 System.out.println();
79 }
80

```

delPos_1009 yang berfungsi untuk menghapus node pada posisi tertentu dalam DLL. Method ini menerima dua parameter: head_1009 dan pos_1009. Pertama kita cek apakah DLL kosong. Lalu kita beralih ke posisi yang diinginkan. Jika posisi tidak valid (melebihi jumlah node), kita return head tanpa perubahan. Jika valid, kita update pointer prev_1009 dari node setelah curr dan next_1009 dari node sebelum curr. Kalau yang dihapus adalah head, kita update head_1009 ke node berikutnya. Ini yang paling kompleks karena harus menangani berbagai kasus pointer.

```

81 public static void main(String[] args) {
82 // create a DLL
83 NodeDLL_2511531009 head_1009 = new NodeDLL_2511531009(1);
84 head_1009.next_1009 = new NodeDLL_2511531009(2);
85 head_1009.next_1009.prev_1009 = head_1009;
86 head_1009.next_1009.next_1009 = new NodeDLL_2511531009(3);
87 head_1009.next_1009.prev_1009 = head_1009.next_1009;
88 head_1009.next_1009.next_1009.next_1009 = new NodeDLL_2511531009(4);
89 head_1009.next_1009.next_1009.prev_1009 = head_1009.next_1009;
90 head_1009.next_1009.next_1009.next_1009.next_1009 = new NodeDLL_2511531009(5);
91 head_1009.next_1009.next_1009.next_1009.next_1009.prev_1009 = head_1009.next_1009.next_1009.next_1009;
92
93 System.out.print("DLL awal : ");
94 printList(head_1009);
95
96 System.out.print("Setelah head dihapus : ");
97 head_1009 = delHead_1009(head_1009);
98 printList(head_1009);
99
100 System.out.print("Setelah last node dihapus : ");
101 head_1009 = delLast_1009(head_1009);
102 printList(head_1009);
103
104 System.out.print("hapus node ke 2 : ");
105 head_1009 = delPos_1009(head_1009, 2);
106 printList(head_1009);
107
108
109
110
111 }
112 }

```

```

DLL awal : 1 2 3 4 5
Setelah head dihapus : 2 3 4 5
Setelah last node dihapus : 2 3 4
hapus node ke 2 : 2 4

```

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Double Linked List (DLL) merupakan pengembangan dari Single Linked List dengan penambahan pointer prev pada setiap node. Hal ini memungkinkan penelusuran dua arah yang lebih fleksibel. Operasi dasar DLL meliputi penelusuran maju dan mundur, penyisipan di awal, akhir, dan posisi tertentu, serta penghapusan dari head, tail, dan posisi tertentu.

Implementasi DLL dalam Java pada praktikum ini menggunakan empat kelas: NodeDLL sebagai definisi node, PenelusuranDLL untuk operasi traversal, insertDLL untuk operasi penyisipan, dan HapusDLL untuk operasi penghapusan. Setiap operasi harus selalu memperbarui pointer next dan prev secara seksama untuk menjaga konsistensi struktur data.

3.2 Saran

Seperti biasa, mahasiswa sebaiknya dapat diarah dengan lebih baik selama praktikum berlangsung dengan metode pembelajaran yang lebih *engaging* dan interaktif, untuk menghindari suasana yang monoton dan canggung.

BAB IV
DAFTAR PUSTAKA

- [1] W3Schools, "DSA Linked Lists Types," *W3Schools.com*. [Online]. Available: https://www.w3schools.com/dsa/dsa_data_linkedlists_types.php. [Accessed: May 14, 2026].

LAPORAN TUGAS PEKAN 6 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2026

Lampiran Tugas

Topik : Implementasi Double Linked List

Tema : Playlist Lagu Sederhana

1. Deskripsi Tugas

Membuatkan program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. Aturan Penamaan

- a) Nama Kelas : concat sufiks “_” + NIM
Contoh : Lagu_2511531009 dan Musik_2511531009
- b) Nama Variabel dan Method : concat sufiks “_” + NIM mod 10^4
Contoh : String judul_1009, tambahLagu_1009
- c) Pointer : next_1009, prev_1009

3. Struktur Program

a. Kelas ADT Pasien (Lagu_2511531009)

Atribut :

- judul_1009 (String)
- penyanyi_1009 (String)
- next_1009 (pointer ke Lagu)
- prev_1009 (pointer ke lagu)

Method :

- Constructor untuk menginisialisasi semua atribut
- Selektor (Getter) untuk semua atribut
- Mutator (Setter) untuk setiap atribut

b. Kelas Driver (Musik_2511531009)

- Daftarkan Lagu (tambahLagu_1009) : Menambahkan lagu baru di AKHIR playlist
- hapusLaguAwal_1009 (Delete Head) : Menghapus node terdepan dan menampilkan data lagu yang dipanggil. Head digeser ke node berikutnya.
- forwardTransveral (tampilMaju_1009) : Menelusuri semua lagu dari awal ke akhir
- backwardTransveral (tampilMundur_1009) : Menampilkan semua lagu dari akhir ke awal (wajib DLL)
- cariLagu_1009(Lagu_2511531009 judul_1009) : Mencari lagu berdasarkan judul (tidak case-sensitive)

4. Contoh Tampilan Output Program

=== Antrian Musik NIM: 2511531009 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1

Judul: Submorphics - Weather It Out (feat. Steo)

Penyanyi: Rosalía

Lagu berhasil ditambahkan!

5. Kriteria Penilaian

1. Pointer next dan prev terhubung dengan benar
2. Aturan penamaan NIM dipatuhi 100%
3. Program tidak error saat playlist kosong
4. Fitur tampil mundur berfungsi
5. Kode rapi dan ada komentar singkat

6. Deadline

Tugas dikumpulkan paling lambat pada hari Rabu, tanggal 13 Mei 2026, pukul 23.59 WIB.

Java (ADT) Class · Lagu_2511531009

Kelas ini menginisialisasikan node dan pointer untuk digunakan pada class driver.

```
1 package pekan6_2511531009;
2
3 public class Lagu_2511531009 {
4
5     // --- Atribut ---
6     private String judul_1009;
7     private String penyanyi_1009;
8     Lagu_2511531009 next_1009; // pointer ke node berikutnya
9     Lagu_2511531009 prev_1009; // pointer ke node sebelumnya
10
11     // --- Constructor ---
12     public Lagu_2511531009(String judul_1009, String penyanyi_1009) {
13         this.judul_1009 = judul_1009;
14         this.penyanyi_1009 = penyanyi_1009;
15         this.next_1009 = null;
16         this.prev_1009 = null;
17     }
18
19     // --- Getter (Selektor) ---
20     public String getJudul_1009() {
21         return judul_1009;
22     }
23
24     public String getPenyanyi_1009() {
25         return penyanyi_1009;
26     }
27
28     // --- Setter (Mutator) ---
29     public void setJudul_1009(String judul_1009) {
30         this.judul_1009 = judul_1009;
31     }
32
33     public void setPenyanyi_1009(String penyanyi_1009) {
34         this.penyanyi_1009 = penyanyi_1009;
35     }
36
37 }
```

Java (Main/Driver) Class · Musik_2511531009

Kelas ini berfungsi sebagai untuk mengelola operasi pada linked list, seperti menambahkan lagu, “menyelesaikan”(atau menghapus) lagu, menampilkan queue lagu, dan mencari lagu

```
1 package pekan6_2511531009;
2
3 import java.util.Scanner;
4
5 public class Musik_2511531009 {
6
7     // Head dan tail playlist
8     private static Lagu_2511531009 head_1009 = null;
9     private static Lagu_2511531009 tail_1009 = null;
10
11     // tambahLagu_1009 : menambahkan lagu baru di AKHIR playlist
12     public static void tambahLagu_1009(String judul_1009, String penyanyi_1009) {
13         Lagu_2511531009 baru_1009 = new Lagu_2511531009(judul_1009, penyanyi_1009);
14
15         if (head_1009 == null) {
16             // Playlist masih kosong : node pertama sekaligus head & tail
17             head_1009 = baru_1009;
18             tail_1009 = baru_1009;
19         } else {
20             // Sambungkan node baru di belakang tail saat ini
21             baru_1009.prev_1009 = tail_1009;
22             tail_1009.next_1009 = baru_1009;
23             tail_1009 = baru_1009;
24         }
25         System.out.println("Lagu berhasil ditambahkan!");
26     }
27
28     // hapusLaguAwal_1009 (Delete Head) : hapus node terdepan
29     public static void hapusLaguAwal_1009() {
30         if (head_1009 == null) {
31             System.out.println("Playlist kosong, tidak ada lagu yang dihapus.");
32             return;
33         }
34
35         System.out.println("Lagu dihapus: \"" + head_1009.getJudul_1009()
36             + "\" oleh " + head_1009.getPenyanyi_1009());
37
38         if (head_1009 == tail_1009) {
39             head_1009 = null;
40             tail_1009 = null;
41         } else {
42             head_1009 = head_1009.next_1009;
43             head_1009.prev_1009 = null;
44         }
45     }
46
47     // tampilMaju_1009 (Forward Traversal) : dari head ke tail
48     public static void tampilMaju_1009() {
49         if (head_1009 == null) {
50             System.out.println("Playlist kosong.");
51             return;
52         }
53
54         System.out.println("--- Playlist (Maju) ---");
55         Lagu_2511531009 current = head_1009;
56         int no = 1;
57         while (current != null) {
58             System.out.println(no + ". " + current.getJudul_1009()
59                 + " - " + current.getPenyanyi_1009());
60             current = current.next_1009;
61             no++;
62         }
63     }
64 }
```

```

65 // tampilMundur_1009 (Backward Traversal) : dari tail ke head
66 public static void tampilMundur_1009() {
67     if (tail_1009 == null) {
68         System.out.println("Playlist kosong.");
69         return;
70     }
71
72     System.out.println("--- Playlist (Mundur) ---");
73     Lagu_2511531009 current_1009 = tail_1009;
74     int no_1009 = 1;
75     while (current_1009 != null) {
76         System.out.println(no_1009 + ". " + current_1009.getJudul_1009()
77             + " - " + current_1009.getPenyanyi_1009());
78         current_1009 = current_1009.prev_1009;
79         no_1009++;
80     }
81 }
82
83 // -----
84 // carilagu_1009 : mencari lagu berdasarkan judul (case-insensitive)
85 // -----
86 public static void carilagu_1009(String judul_1009) {
87     if (head_1009 == null) {
88         System.out.println("Playlist kosong.");
89         return;
90     }
91
92     Lagu_2511531009 current_1009 = head_1009;
93     boolean ditemukan_1009 = false;
94     int no = 1;
95
96     while (current_1009 != null) {
97         if (current_1009.getJudul_1009().equalsIgnoreCase(judul_1009)) {
98             System.out.println("Lagu ditemukan (#" + no + "): \""
99                 + current_1009.getJudul_1009() + "\" oleh "
100                 + current_1009.getPenyanyi_1009());
101             ditemukan_1009 = true;
102         }
103         current_1009 = current_1009.next_1009;
104         no++;
105     }
106
107     if (!ditemukan_1009) {
108         System.out.println("Lagu \"" + judul_1009 + "\" tidak ditemukan.");
109     }
110 }
111
112 // -----
113 // Main : menu interaktif
114 // -----
115 public static void main(String[] args) {
116     Scanner sc_1009 = new Scanner(System.in);
117     int pilihan_1009 = 0;
118
119     do {
120         System.out.println("\n=== Antrian Musik NIM: 2511531009 ===");
121         System.out.println("1. Tambah Lagu");
122         System.out.println("2. Hapus Lagu Pertama");
123         System.out.println("3. Lihat Playlist (Maju)");
124         System.out.println("4. Lihat Playlist (Mundur)");
125         System.out.println("5. Cari Lagu");
126         System.out.println("6. Keluar");
127         System.out.print("Pilihan: ");
128
129         if (sc_1009.hasNextInt()) {
130             pilihan_1009 = sc_1009.nextInt();
131         } else {
132             sc_1009.next();
133             pilihan_1009 = -1;
134         }
135         sc_1009.nextLine();
136
137         switch (pilihan_1009) {
138             case 1:
139                 System.out.print("Judul : ");
140                 String judul_1009 = sc_1009.nextLine();
141                 System.out.print("Penyanyi: ");
142                 String penyanyi_1009 = sc_1009.nextLine();
143                 tambahLagu_1009(judul_1009, penyanyi_1009);
144                 break;
145             case 2:
146                 hapusLaguAwal_1009();
147                 break;
148             case 3:
149                 tampilMaju_1009();
150                 break;
151             case 4:
152                 tampilMundur_1009();
153                 break;
154             case 5:
155                 System.out.print("Masukkan judul lagu: ");
156                 String cari_1009 = sc_1009.nextLine();
157                 carilagu_1009(cari_1009);
158                 break;
159             case 6:
160                 System.out.println("Terima kasih! Program selesai.");
161                 break;
162             default:
163                 System.out.println("Pilihan tidak valid. Silakan coba lagi.");
164         }
165     } while (pilihan_1009 != 6);
166
167     sc_1009.close();
168 }
169
170 }
171

```

```

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul : From the Start
Penyanyi: Laufey
Lagu berhasil ditambahkan!

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul : The Perfect Pair
Penyanyi: Beabadoobee
Lagu berhasil ditambahkan!

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul : Babydoll
Penyanyi: Dominic Fike
Lagu berhasil ditambahkan!

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
--- Playlist (Maju) ---
1. From the Start - Laufey
2. The Perfect Pair - Beabadoobee
3. Babydoll - Dominic Fike

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
--- Playlist (Mundur) ---
1. Babydoll - Dominic Fike
2. The Perfect Pair - Beabadoobee
3. From the Start - Laufey

```

```

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: babydoll
Lagu ditemukan (#3): "Babydoll" oleh Dominic Fike

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: doraemon
Lagu "doraemon" tidak ditemukan.

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu dihapus: "From the Start" oleh Laufey

=== Antrian Musik NIM: 2511531009 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
--- Playlist (Maju) ---
1. The Perfect Pair - Beabadoobee
2. Babydoll - Dominic Fike

```

```
1. Babydoll - Dominic Fike  
=== Antrian Musik NIM: 2511531009 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 4  
--- Playlist (Mundur) ---  
1. Babydoll - Dominic Fike  
2. The Perfect Pair - Beabadoobee  
=== Antrian Musik NIM: 2511531009 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 7  
Pilihan tidak valid. Silakan coba lagi.  
=== Antrian Musik NIM: 2511531009 ===  
1. Tambah Lagu  
2. Hapus Lagu Pertama  
3. Lihat Playlist (Maju)  
4. Lihat Playlist (Mundur)  
5. Cari Lagu  
6. Keluar  
Pilihan: 6  
Terima kasih! Program selesai.
```